First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is allocated or not.
- If allocated, it displays the process number, process size, block number, block size,

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int block[m], original[m];
    printf("Enter size of each block:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        original[i] = block[i]; // store original size
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int process[n];
    printf("Enter size of each process:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &process[i]);
    }
```

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is allocated or not.
- If allocated, it displays the process number, process size, block number, block size, and the internal fragmentation (block size minus process size).

Select Language : C (GCC 9.2.0)

```
26      for (int i = 0; i < n; i++) {
27          int allocated = 0;
28
29          for (int j = 0; j < m; j++) {
30              if (block[j] >= process[i]) {
31                  int fragment = block[j] - process[i];
32
33                  printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
34                         i + 1, process[i], j + 1, original[j], fragment);
35
36                  block[j] = fragment; // reduce available size
37                  allocated = 1;
38                  break;
39              }
40          }
41
42          if (!allocated) {
43              printf("Process %d of size %d is not allocated\n", i + 1, process[i]);
44          }
45      }
46
47      return 0;
48  }
```

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 2 of size 500 with Fragment 288
Process 2 of size 417 is allocated to Block 5 of size 600 with Fragment 183
Process 3 of size 112 is allocated to Block 2 of size 500 with Fragment 176
Process 4 of size 426 is not allocated
```

Best Fit Contiguous Memory Allocation

Completed

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int block[m], remainingSize[m];

    printf("Enter size of each block:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        remainingSize[i] = block[i];   // initialize remaining memory
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int process[n];

    printf("Enter size of each process:\n");
    for(int i = 0; i < n; i++) {
```

Best Fit Contiguous Memory Allocation

Completed

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not

Select Language : C (GCC 9.2.0)

```c
            if(remainingSize[j] >= process[i]) {
                if(best == -1 || remainingSize[j] < remainingSize[best]) {
                    best = j;
                }
            }
        }

        if(best != -1) {
            int fragment = remainingSize[best] - process[i];

            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                   i + 1, process[i], best + 1, block[best], fragment);

            remainingSize[best] -= process[i];   // update remaining memory
        }
        else {
            printf("Process %d of size %d is not allocated\n",
                   i + 1, process[i]);
        }
    }

    return 0;
}
```

Input :

```
5
100 500 200 300 600
4
212 417 112 426
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174
```

Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 212 is allocated to Block 4 of size 300 with Fragment 88
Process 2 of size 417 is allocated to Block 2 of size 500 with Fragment 83
Process 3 of size 112 is allocated to Block 3 of size 200 with Fragment 88
Process 4 of size 426 is allocated to Block 5 of size 600 with Fragment 174
```